

FinSight AI Commerce — Interview Guide

Document Name: How to Explain It in Interviews | **Author:** Full-Stack & AI Engineer

■ The 90-Second 'Sweet Spot' Interview Pitch

Deliver this script when asked: 'Tell me about your project' or 'Walk me through a project you built.'

"Have you ever tried searching an e-commerce site for something specific—like 'quiet headphones for coding in coffee shops under \$200'—only to get zero results or random irrelevant products?

That happens because standard search engines only look for exact keywords. They can't process human context or budget caps together.

To solve this, I built **FinSight AI Commerce**—an intelligent full-stack shopping platform.

Here is how I engineered it:

- **Understanding Context:** When a user types a prompt, my FastAPI backend uses **Llama 3 (via Groq)** to break the query down into structured intent, target use-cases, and exact price limits.
- **Smart Retrieval:** Instead of basic SQL search, I used **ChromaDB vector search** to find products by *meaning* rather than exact words, while enforcing hard metadata rules so budget limits are 100% strictly respected.
- **Financial Value:** The UI shows custom **'Investment Tiers'** (*Budget Pick, Mid-Range, Premium*) and an AI pitch explaining *why* a product is worth your money.

The biggest challenge was making sure the AI never hallucinated prices or broke search constraints. I solved this by engineering a **dual validation pipeline** with regex fallbacks and Pydantic schema validation.

*I built the whole system end-to-end using **Next.js, Python, FastAPI, Docker, and GitHub Actions**, achieving sub-1 second search speeds."*

■ Why This Pitch Connects with Interviewers

What You Say	What the Interviewer Hears
'Have you ever tried searching...?'	Hook & Relatability: Immediately engages the interviewer with a common real-world frustration.
'Llama 3 + ChromaDB Vector Search'	Modern AI Expertise: Demonstrates production RAG (Retrieval-Augmented Generation) knowledge.
'Enforcing hard metadata rules'	Production Mindset: Proves you don't blindly trust LLMs; you sanitize & control AI outputs.
'FastAPI, Next.js, Docker, CI/CD'	Full-Stack Competency: Shows you can build, containerize, and ship robust software.

■ Comprehensive Interview Q&A; Guide

Be prepared to answer these follow-up technical questions confidently:

Q1: Why use a Vector Database (ChromaDB) instead of traditional SQL full-text search?

SQL full-text search requires exact string matching. If a user searches for 'headphones for noisy flights', SQL misses products described as 'noise-cancelling travel headsets'. ChromaDB converts text into 384-dimensional vector embeddings, allowing vector similarity search based on semantic meaning rather than literal words.

Q2: How does your Hybrid Search combine vector similarity with numerical filtering?

Vector search alone cannot guarantee hard scalar constraints (e.g. price $\leq$ \$200). In my backend, I pass a metadata filter dictionary into ChromaDB: ``where={'price': {'$lte': max_budget}}``. This filters out non-compliant products first, then ranks the remaining items by vector similarity.

Q3: How do you handle LLM format errors or hallucinations?

I built a two-tier safety net: (1) Strict Pydantic schema parsing (``CustomerIntent`` model), and (2) a regex-based fallback extraction engine. If Groq returns non-JSON or extra text, the regex engine extracts ``budget``, ``category``, and ``use_case`` directly from the raw string. If the API fails completely, standard text retrieval runs without throwing a 500 server error.

Q4: Why FastAPI instead of Flask or Django?

FastAPI leverages Python's native ``async/await`` syntax for high concurrency, auto-generates interactive Swagger API docs (/docs), and natively integrates with Pydantic for high-performance request/response data validation.

Q5: Why Groq / Llama 3 for LLM inference?

Groq's LPU (Language Processing Unit) architecture delivers ultra-low inference latency (~500+ tokens/sec). This allows our intent extraction step to execute in under 200ms, keeping total end-to-end API response times under 1 second.

Q6: How is the frontend structured in Next.js 16?

The frontend uses Next.js App Router with TypeScript and Tailwind CSS. It features a responsive grid, real-time intent badge indicators, slide-out shopping cart drawer with live subtotal calculation, and money category filters (Budget Pick, Mid-Range, Premium, Investment Grade).

Q7: How did you containerize and deploy this application?

The app uses Docker Compose to orchestrate two isolated containers: (1) FastAPI Python 3.11 backend service running uvicorn, and (2) Next.js Node environment frontend service. GitHub Actions automatically builds and verifies both containers on every git push.